UNIVERSIDAD PRIVADA DEL NORTE (UPN)

FUNDAMENTOS DE PROGRAMACIÓN

DESAFÍO DEL CURSO — INFORME TÉCNICO

MESA DE PARTES DIGITAL- SISMO

Prototipo para la gestión de solicitudes ciudadanas ante un terremoto de gran magnitud en Lima Metropolitana

Integrantes:

Elber Nuñez Iguavel	N00535064
Milovan Rojas Romero	N00522750
Angel Rafael Caldas Ocmin	N00541516

Docente: Williams Alfonso Chullo Villagaray

OCTUBRE-2026

INTRODUCCIÓN

Ante un terremoto de gran magnitud en Lima Metropolitana, una municipalidad puede recibir numerosas solicitudes ciudadanas: reportes de daños, pedidos de ayuda, solicitudes de evaluación y requerimientos de atención. Si el registro se realiza manualmente o mediante canales dispersos, pueden aparecer demoras, duplicidad de registros, pérdida de información y dificultades para identificar qué casos requieren atención prioritaria.

El proyecto “Mesa de Partes Digital–Sismo” propone analizar el proceso de recepción y registro de solicitudes y definir los requerimientos de un prototipo que permita registrar, validar, clasificar, buscar y almacenar expedientes. La clasificación preliminar considerada por el tema elegido contempla cuatro niveles: crítico, alto, medio y bajo. Los criterios definitivos de clasificación deberán ser acordados con el responsable municipal y no deben sustituir la evaluación de los equipos de emergencia.

Este informe desarrolla únicamente los pasos 1 y 2 de la práctica de campo indicada en la guía: (1) observación del proceso real y (2) levantamiento de requerimientos. Para mantener la veracidad académica, los cuadros de observación que siguen son una plantilla de trabajo y un modelo preliminar; el equipo debe completar o corregir los datos con una visita, entrevista o evidencia real antes de presentarlos como hechos observados.

Objetivo del informe

Documentar el proceso actual de recepción de solicitudes ciudadanas y analizar sus problemas, entradas, transformaciones y salidas, con el fin de establecer requerimientos funcionales y no funcionales para un prototipo de Mesa de Partes Digital–Sismo.

Alcance

Se incluyen solamente la observación del proceso y el levantamiento de requerimientos. No se desarrolla todavía la comparación de alternativas, el diseño del algoritmo ni la implementación.

PASO 01: OBSERVACIÓN DEL PROCESO REAL

Propósito de la observación

La observación del proceso real tiene como finalidad conocer cómo una oficina de mesa de partes recibe y registra documentos o solicitudes, qué información utiliza, qué actividades realiza el personal y cuánto tiempo requiere cada etapa.

Esta observación permitirá comprender cómo funciona el procedimiento actual y reconocer las dificultades que podrían aumentar durante una emergencia sísmica, cuando la cantidad de solicitudes ciudadanas puede incrementarse considerablemente.

Datos de la observación:

Entidad observada: Municipalidad Distrital de Santa Anita

Área: Mesa de Partes o Trámite Documentario

Lugar: Av. Los Eucaliptos, Santa Anita (frente a la Plaza de Armas del distrito o muy cerca del óvalo de Santa Anita)

Fecha de observación: 19/10/2026

Hora de inicio y finalización: 11:36 AM, 11:54AM

Evidencias recopiladas

PROCESO

Durante la observación identificamos cómo se desarrolla el procedimiento de recepción y registro de solicitudes. Para organizar la información, se consideran las siguientes actividades que deben verificarse en la institución seleccionada:

1.- Recepción de la solicitud.

El ciudadano presenta un documento o comunica una necesidad mediante alguno de los canales habilitados por la institución. Se debe identificar qué tipos de solicitudes se reciben y qué datos se solicitan inicialmente. Anexo-01

2.- Revisión de la información.

El personal verifica si la solicitud contiene los datos y documentos requeridos se debe observar cómo se identifican los campos incompletos, los errores y los documentos faltantes. Anexo-02

3.- Registro de la solicitud.

La información se registra mediante el mecanismo utilizado por la institución, como un sistema informático, un formulario o un registro físico. Anexo-03

4.- Derivación al área responsable.

La solicitud se remite al área encargada de revisarla o atenderla, verificamos cómo se registra la derivación y cómo se puede consultar posteriormente el estado del expediente. Anexo-04

5.- Confirmación y seguimiento.

Identificaremos si el ciudadano recibe una constancia, un número de expediente u otro mecanismo de seguimiento. También se debe observar cómo se consulta la información registrada.

Nuestra Bitácora de observación

La siguiente tabla permite registrar las actividades, los tiempos y las dificultades identificadas durante nuestra visita a Municipalidad Distrital de Santa Anita.

N.º	Actividad	Formato o información utilizada	Tiempo observado	Problemas o evidencias
1	Recepción de la solicitud ciudadana.	Anexo-01	11:40 am 4 minutos	Acumulación de solicitudes durante una emergencia y demora en la atención inicial.
2	Revisión de los datos y documentos.	Anexo-02	11:45 am 5 minutos	Datos incompletos, errores en la información o documentos faltantes
3	Registro de la solicitud y asignación del expediente.	Anexo-03	11:49 am 4 minutos	Posibles errores de digitación, duplicidad de registros o demora en asignar el número de expediente.
4	Derivación al área responsable.	Anexo-04	11:52 am 3 minutos	Demora en enviar la solicitud al área correspondiente o falta de

				actualización sobre su derivación.
5	Confirmación y consulta del estado.	Anexo-05	11:54 am 2 minutos	Dificultad para consultar el estado del expediente y brindar información actualizada al ciudadano.

Para analizar el funcionamiento de la mesa de partes se identifican tres componentes principales:

Entradas: son los datos y documentos que ingresan al proceso. En el prototipo propuesto podrían incluir el tipo de solicitud, la descripción del problema, la fecha, la ubicación referencial, los datos mínimos del solicitante y los documentos adjuntos.

Procesos: son las actividades que transforman la información recibida. Entre ellas podrían encontrarse la validación de los datos, el registro del expediente, la asignación de un código, la clasificación preliminar de la solicitud y su derivación al área correspondiente.

Salidas: son los resultados generados por el proceso, como un expediente registrado, un código de seguimiento, el estado de la solicitud, el nivel de prioridad preliminar y una constancia de recepción.

Justificación de su Elección

Elegimos este proceso porque la Mesa de Partes cumple una función importante en la recepción y gestión de documentos y solicitudes de los ciudadanos. Ante un terremoto de gran magnitud en Lima Metropolitana, la cantidad de reportes de daños y pedidos de ayuda podría aumentar considerablemente, haciendo necesario organizar y registrar la información de manera rápida y ordenada.

Por ello, consideramos importante analizar este proceso para identificar posibles dificultades en la recepción, revisión, registro y seguimiento de los expedientes. Esta observación servirá como base para proponer el prototipo Mesa de Partes Digital–Sismo, que permitirá registrar solicitudes, validar datos, clasificarlas preliminarmente por nivel de prioridad y facilitar la búsqueda de expedientes.

PASO 2: LEVANTAMIENTO DE REQUERIMIENTOS

Análisis del problema mediante entradas, procesos y salidas

A partir de la información que se recopile en la observación y la entrevista, se analizarán los elementos necesarios para el funcionamiento del prototipo.

Elemento	**Necesidad identificada o por validar**	**Requerimiento para el prototipo**
Entradas	Los datos de las solicitudes deben registrarse de manera uniforme.	Crear un formulario con campos definidos para cada expediente.
Validación	Es necesario detectar campos obligatorios incompletos.	Verificar los datos antes de guardar la solicitud.
Registro	Se requiere identificar cada expediente de forma individual.	Generar un código único para cada solicitud registrada.
Clasificación	Se necesita diferenciar las solicitudes según su prioridad.	Incorporar una clasificación preliminar en cuatro niveles: crítico, alto, medio y bajo.
Búsqueda	El personal necesita localizar los expedientes registrados.	Permitir búsquedas por código y otros criterios disponibles.
Seguimiento	Es necesario conocer el estado de cada solicitud.	Registrar y consultar el estado y el área responsable.
Almacenamiento	La información debe conservarse para consultas posteriores.	Guardar los expedientes mediante un mecanismo de almacenamiento apropiado para el prototipo.
Seguridad	Los datos ciudadanos deben manejarse de forma responsable.	Limitar la exposición de datos y controlar las acciones sobre la información.

Elemento	Descripción

Entradas	Datos del ciudadano, tipo de solicitud, descripción del problema, ubicación y documentos adjuntos.
Procesos	Recibir la solicitud, revisar los datos, registrar el expediente, asignar un código y derivarlo al área responsable.
Salidas	Solicitud registrada, código de expediente, nivel de prioridad y estado de la solicitud.

Problema identificado: El aumento de solicitudes durante una emergencia puede dificultar su registro, organización y seguimiento.

Solución propuesta: Crear un prototipo digital que permita registrar, clasificar y consultar las solicitudes **de manera ordenada.**

Problemas frecuentes del registro manual

El registro manual de solicitudes ciudadanas puede presentar las siguientes dificultades:

N.º	Problema	Consecuencia
1	Errores al escribir los datos.	Información incorrecta en el expediente.
2	Datos incompletos o documentos faltantes.	Retrasos en el registro de la solicitud.
3	Registro duplicado de solicitudes.	Confusión y repetición de información.
4	Demora en buscar expedientes.	Mayor tiempo de atención al ciudadano.
5	Dificultad para actualizar el estado de la solicitud.	Falta de información actualizada sobre su seguimiento.

Análisis del problema

Estas dificultades podrían aumentar si ocurre un terremoto de gran magnitud y se reciben numerosas solicitudes ciudadanas en poco tiempo. Por ello, proponemos desarrollar un prototipo en Python que permita registrar datos, validar campos obligatorios, asignar códigos a los expedientes y facilitar su búsqueda y seguimiento.

PASO 03: ANÁLISIS DE ALTERNATIVAS DE SOLUCIÓN

3.1. Identificación de alternativas

A partir de los problemas identificados en los pasos anteriores, como errores de registro, duplicidad de solicitudes y demoras en la búsqueda de expedientes, se proponen tres alternativas para mejorar la gestión de solicitudes ciudadanas ante un terremoto de gran magnitud en Lima Metropolitana.

Alternativa 1: Desarrollo de un prototipo en Python.
Implementar un programa que permita registrar, validar, clasificar preliminarmente, buscar y almacenar expedientes mediante archivos. Se utilizarán funciones, listas, cadenas de caracteres y estructuras de control, gestionando los avances con Git y GitHub.

Alternativa 2: Automatización mediante una hoja de cálculo.
Utilizar Excel o Google Sheets para organizar las solicitudes mediante tablas, filtros, fórmulas y validaciones. Es una opción sencilla, aunque puede presentar limitaciones para automatizar procesos más complejos.

Alternativa 3: Tercerización del sistema.
Contratar a una empresa o profesional externo para desarrollar el sistema según los requerimientos municipales. Sin embargo, requiere presupuesto y coordinación con el proveedor, además de considerar los costos de mantenimiento y futuras modificaciones.

3.2. Cuadro comparativo de las alternativas

Criterio	Alternativa 1: Python	Alternativa 2: Excel Hoja de cálculo	Alternativa 3: Tercerización
Requerimientos funcionales	Permite integrar registro, validación, clasificación preliminar, búsqueda y almacenamiento.	Permite registrar, filtrar y validar datos con funciones disponibles.	Puede integrar funcionalidades personalizadas según el contrato.
Recursos necesarios	Computadora, Python, editor de código y Git/GitHub.	Computadora y herramienta de hojas de cálculo.	Presupuesto, proveedor y recursos para supervisar el desarrollo.
Costo inicial	Bajo, aprovechando herramientas gratuitas y los conocimientos del curso.	Bajo si se dispone de una herramienta adecuada.	Depende de la cotización y del alcance contratado.
Facilidad de implementación	Requiere diseñar el algoritmo y programar sus funciones.	Es relativamente sencilla para operaciones básicas.	Requiere definir requisitos, contratar y coordinar con el proveedor.
Posibilidades de ampliación	Permite incorporar nuevas funciones y mejorar la estructura del programa.	Depende de las capacidades de la herramienta y de la complejidad de las automatizaciones.	Depende del contrato, la tecnología elegida y los costos de modificación.
Viabilidad para el proyecto	Alta, porque responde a los requerimientos del desafío y permite aplicar los conceptos del curso.	Media, porque resuelve parte de las necesidades, pero puede presentar limitaciones al aumentar la complejidad.	Media o baja para este alcance académico, debido a los costos y a la dependencia de un proveedor.

Nota: La valoración es cualitativa y corresponde al análisis preliminar del equipo. No se cuenta con cotizaciones económicas verificadas para la alternativa de tercerización.

3.3. Selección y justificación de la solución

Se selecciona el desarrollo de un prototipo en Python porque permite atender los requerimientos identificados en el paso 2, utilizando los conocimientos adquiridos en el curso.

El sistema contará con un menú para registrar, validar, buscar y ordenar expedientes, así como almacenar la información en archivos. Además, permitirá clasificar preliminarmente las solicitudes en cuatro niveles: crítico, alto, medio y bajo. Los criterios de clasificación deberán ser validados por el responsable municipal y no reemplazarán la evaluación de los equipos de emergencia.

Esta alternativa es viable porque utiliza recursos accesibles, permite reutilizar funciones y facilita futuras mejoras sin asumir inicialmente los costos de tercerización. Así, se busca reducir los errores y las demoras, mejorando la organización y consulta de las solicitudes ciudadanas.

PASO 04: DISEÑO DEL ALGORITMO

4.1. Descripción del diseño propuesto

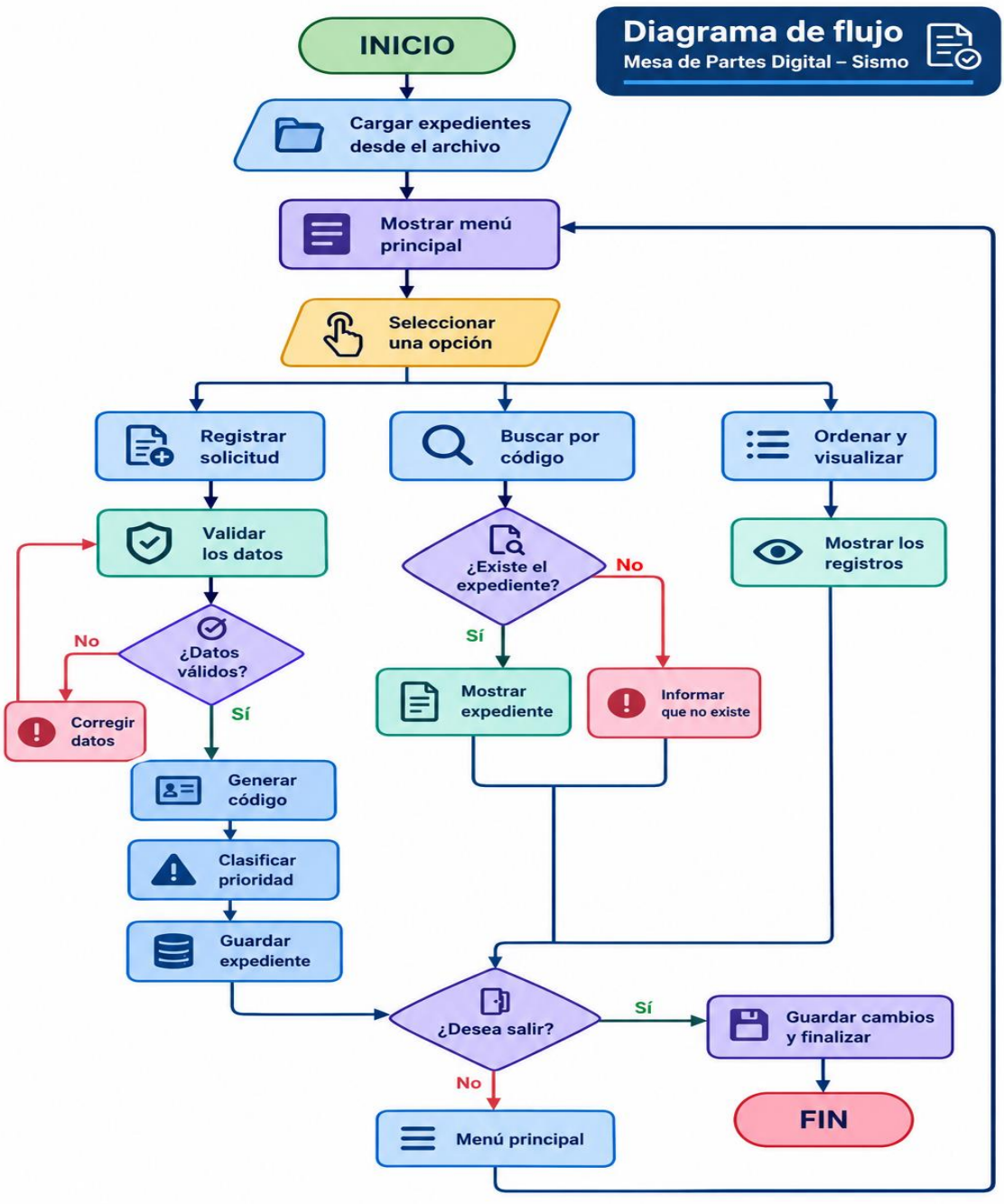
El prototipo *Mesa de Partes Digital–Sismo* se organizará mediante un menú principal que permitirá registrar, buscar y ordenar expedientes, así como salir del sistema.

Al iniciar, se cargarán los expedientes guardados previamente. Para registrar una solicitud, se ingresarán los datos del ciudadano, el tipo de solicitud, la descripción del problema y la ubicación. El sistema validará la información y, si es correcta, generará un código único, asignará una prioridad preliminar y guardará el expediente en un archivo.

La búsqueda permitirá consultar los datos mediante el código del expediente e informar si no se encuentra. Asimismo, los registros podrán visualizarse ordenados por código. El menú se repetirá hasta que el usuario decida salir, guardando los cambios para conservar la información.

4.2. Diagrama de flujo del algoritmo

El siguiente diagrama representa la secuencia lógica propuesta para el funcionamiento del prototipo.



Interpretación del diagrama: el flujo inicia con la carga de los expedientes y la presentación del menú. Según la opción seleccionada, se realiza el registro, la búsqueda o la visualización ordenada. Para registrar una solicitud, es obligatorio validar los datos antes de generar el código y guardar el expediente. El ciclo se mantiene hasta que el usuario decide salir.

4.3. Pseudocódigo del algoritmo

El pseudocódigo permite describir el funcionamiento del sistema antes de implementar el código en Python. Para facilitar su comprensión, se utilizan instrucciones secuenciales, condicionales y repetitivas, además de procedimientos que representan las funciones principales del prototipo.

```
ALGORITMO MesaDePartesDigitalSismo

    DEFINIR expedientes COMO LISTA
    DEFINIR opcion COMO ENTERO

    expedientes ← CARGAR_EXPEDIENTES_DESDE_ARCHIVO()

    REPETIR

        MOSTRAR "MESA DE PARTES DIGITAL - SISMO"
        MOSTRAR "1. Registrar solicitud"
        MOSTRAR "2. Buscar expediente por código"
        MOSTRAR "3. Ordenar y visualizar expedientes"
        MOSTRAR "4. Salir"

        LEER opcion

        SEGÚN opcion HACER
            CASO 1:
                solicitud ← INGRESAR_DATOS()

                SI VALIDAR_DATOS(solicitud) = VERDADERO ENTONCES
                    solicitud.codigo ← GENERAR_CODIGO(expedientes)

                    solicitud.prioridad ←
                        CLASIFICAR_PRIORIDAD(solicitud)

                    solicitud.estado ← "Registrado"

                    AGREGAR solicitud A expedientes

                    GUARDAR_EXPEDIENTES(expedientes)

                    MOSTRAR "Solicitud registrada correctamente"
                    MOSTRAR "Código: ", solicitud.codigo
                    MOSTRAR "Prioridad: ", solicitud.prioridad

                SINO
                    MOSTRAR "Existen datos incompletos o inválidos"
                    MOSTRAR "Revise la información ingresada"
                FIN SI

            CASO 2:
```

```
                MOSTRAR "Ingrese el código del expediente"
                LEER codigoBuscado
                resultado ← BUSCAR_EXPEDIENTE(
                    expedientes, codigoBuscado)


                SI resultado EXISTE ENTONCES
                    MOSTRAR resultado
                SINO
                    MOSTRAR "Expediente no encontrado"
                FIN SI

            CASO 3:
                ORDENAR expedientes POR codigo
                MOSTRAR expedientes

            CASO 4:
                GUARDAR_EXPEDIENTES(expedientes)
                MOSTRAR "Finalizando el sistema"

            DE OTRO MODO:
                MOSTRAR "Opción inválida"

        FIN SEGÚN

    HASTA QUE opcion = 4

FIN ALGORITMO
```

Consideraciones del pseudocódigo:

- La función `CARGAR_EXPEDIENTES_DESDE_ARCHIVO()` recupera los registros guardados anteriormente. Si el archivo no existe, el sistema deberá iniciar con una lista vacía.
- La función `VALIDAR_DATOS()` verifica que los campos obligatorios estén completos y cumplan las reglas establecidas.
- La función `GENERAR_CODIGO()` asigna un identificador único para cada expediente, evitando reutilizar códigos existentes.
- La función `CLASIFICAR_PRIORIDAD()` asigna una de las cuatro categorías — crítico, alto, medio o bajo— según los criterios previamente definidos y validados por el responsable municipal.
- Las funciones de búsqueda y ordenamiento facilitan la localización y organización de los registros.
- La función `GUARDAR_EXPEDIENTES()` conserva la información en un archivo para permitir su recuperación en futuras ejecuciones.

4.4. Organización funcional del prototipo

Para facilitar la comprensión, el mantenimiento y la reutilización del código, el prototipo se organizará en funciones que cumplan responsabilidades específicas. Esta organización permitirá corregir o mejorar una funcionalidad sin tener que modificar todo el programa.

Función o módulo	Responsabilidad	Conceptos de programación aplicados
mostrar_menu()	Presentar las opciones disponibles y orientar la interacción con el usuario.	Salida de datos, estructuras de control.
ingresar_datos()	Solicitar y organizar los datos de una nueva solicitud.	Entrada de datos, cadenas y listas.
validar_datos()	Comprobar que los campos obligatorios sean correctos antes de registrar el expediente.	Condicionales, operadores lógicos y cadenas.
generar_codigo()	Crear un código único para identificar cada expediente.	Funciones, variables y operaciones con números o cadenas.
clasificar_prioridad()	Asignar la clasificación preliminar según las reglas definidas.	Condicionales y lógica algorítmica.
registrar_expediente()	Incorporar una solicitud validada a la lista de expedientes.	Listas, funciones y estructuras de control.
buscar_expediente()	Localizar un expediente a partir de su código.	Recorridos de listas, comparaciones y cadenas.
ordenar_expedientes()	Organizar los registros por código para facilitar su consulta.	Listas y algoritmos de ordenamiento.
cargar_expedientes()	Recuperar los registros guardados en ejecuciones anteriores.	Archivos y manejo de datos.
guardar_expedientes()	Almacenar los expedientes para conservar la información.	Archivos y persistencia de datos.

Las funciones se integrarán en un programa principal que controlará el menú y ejecutará las operaciones seleccionadas. Se utilizarán archivos CSV para almacenar y recuperar los expedientes, listas para organizar los registros, cadenas de caracteres para manejar los datos y estructuras condicionales y repetitivas para controlar el funcionamiento del sistema.

4.5. Relación entre el diseño y los requerimientos identificados

El diseño del algoritmo responde a los requerimientos establecidos en el paso 2 de la siguiente manera:

Problema o necesidad identificada	Respuesta del algoritmo
Errores y datos incompletos.	Validación de campos obligatorios antes del registro.
Duplicidad o dificultad para identificar solicitudes.	Generación de códigos únicos para los expedientes.
Necesidad de priorizar la atención.	Clasificación preliminar en cuatro niveles de prioridad.
Demora en localizar expedientes.	Búsqueda por código de identificación.
Dificultad para organizar los registros.	Ordenamiento de expedientes por código.
Riesgo de perder información.	Almacenamiento y recuperación mediante archivos.
Dificultad para hacer seguimiento.	Registro del estado de cada solicitud para facilitar su consulta.

En conclusión, el diseño propuesto establece una secuencia lógica para transformar los datos ingresados por el usuario en expedientes organizados y consultables. La separación del programa en funciones facilitará la implementación progresiva, las pruebas y el mantenimiento del prototipo. Este diseño servirá como base para el paso 5, en el que se implementará la solución en Python utilizando control de versiones mediante Git y GitHub.

PASO 05: IMPLEMENTACIÓN VERSIONADA

Implementación en Python

Se implementó la alternativa elegida en el paso 3 usando solo la biblioteca estándar de Python 3. Se respetó la organización funcional del paso 4, agrupando las funciones en cuatro módulos:

Módulo	Funciones	Responsabilidad
main.py	mostrar_menu(), ingresar_datos(), main()	Menú repetitivo hasta salir
validacion.py	validar_datos()	Campos obligatorios y formatos
expedientes.py	generar_codigo(), clasificar_prioridad(), registrar_expediente(), buscar_expediente(), ordenar_expedientes(), buscar_duplicado()	Reglas del proceso
almacenamiento.py	cargar_expedientes(), guardar_expedientes()	Persistencia en CSV

El código de expediente usa el formato observado en Santa Anita (MOSA-AAAA-NNNNNN, Anexo-05) y el área responsable se asigna según el tipo de solicitud (actividad 4 de la bitácora).

La prioridad (Crítico, Alto, Medio, Bajo) es preliminar y debe ser validada por el responsable municipal.

Capturas del código

Validación (validacion.py)

```
def validar_datos(solicitud):
    """Comprueba los campos obligatorios y devuelve una lista de errores.

    Si la lista está vacía, la solicitud es válida.
    """
    errores = []

    nombres = solicitud.get("nombres", "").strip()
    if len(nombres) < 3:
        errores.append("Nombres y apellidos: obligatorio (mínimo 3 caracteres).")
    elif any(c.isdigit() for c in nombres):
        errores.append("Nombres y apellidos: no debe contener números.")

    dni = solicitud.get("dni", "").strip()
    if not (solo_digitos(dni) and len(dni) == 8):
        errores.append("DNI: debe tener exactamente 8 dígitos.")

    telefono = solicitud.get("telefono", "").strip()
    if not (solo_digitos(telefono) and len(telefono) == 9):
        errores.append("Teléfono: debe tener 9 dígitos.")

    if solicitud.get("tipo", "") not in TIPOS_SOLICITUD:
        errores.append("Tipo de solicitud: seleccione una opción del 1 al 6.")

    descripcion = solicitud.get("descripcion", "").strip()
    if len(descripcion) < 10:
        errores.append("Descripción: obligatoria (mínimo 10 caracteres).")

    ubicacion = solicitud.get("ubicacion", "").strip()
    if len(ubicacion) < 5:
        errores.append("Ubicación: obligatoria (mínimo 5 caracteres).")

    return errores
```

Prioridad preliminar (expedientes.py)

```
def clasificar_prioridad(solicitud):
    """Clasificación PRELIMINAR en cuatro niveles: Crítico, Alto, Medio, Bajo."""
```

La prioridad (Crítico, Alto, Medio, Bajo) es preliminar y debe ser validada por el responsable municipal.

5.2. Capturas del código

Validación (validacion.py)

```
def validar_datos(solicitud):
    """Comprueba los campos obligatorios y devuelve una lista de errores.

    Si la lista está vacía, la solicitud es válida.
    """
```

```
    errores = []

    nombres = solicitud.get("nombres", "").strip()
    if len(nombres) < 3:
        errores.append("Nombres y apellidos: obligatorio (mínimo 3 caracteres).")
    elif any(c.isdigit() for c in nombres):
        errores.append("Nombres y apellidos: no debe contener números.")

    dni = solicitud.get("dni", "").strip()
    if not (solo_digitos(dni) and len(dni) == 8):
        errores.append("DNI: debe tener exactamente 8 dígitos.")

    telefono = solicitud.get("telefono", "").strip()
    if not (solo_digitos(telefono) and len(telefono) == 9):
        errores.append("Teléfono: debe tener 9 dígitos.")

    if solicitud.get("tipo", "") not in TIPOS_SOLICITUD:
        errores.append("Tipo de solicitud: seleccione una opción del 1 al 6.")

    descripcion = solicitud.get("descripcion", "").strip()
    if len(descripcion) < 10:
        errores.append("Descripción: obligatoria (mínimo 10 caracteres).")

    ubicacion = solicitud.get("ubicacion", "").strip()
    if len(ubicacion) < 5:
        errores.append("Ubicación: obligatoria (mínimo 5 caracteres).")

    return errores
```

Prioridad preliminar (expedientes.py)

```
def clasificar_prioridad(solicitud):
    """Clasificación PRELIMINAR en cuatro niveles: Crítico, Alto, Medio, Bajo."""
```

PASO 06 PRUEBAS CON DATOS REALES Y REFLEXIÓN

Casos de prueba

Se usaron datos de la observación del 19/10/2026 en la Municipalidad de Santa Anita (formato MOSA-2026-001256, dirección Av. Los Eucaliptos y los 5 problemas de la bitácora). Los DNI y teléfonos son ficticios. Se ejecutaron 22 pruebas automatizadas (python -m unittest discover -s pruebas -v) y todas fueron aprobadas.

Casos	Actividad / problema observado	Qué se verificó	Resultado
CP01–CP06	11:45 Datos incompletos o erróneos	Campos vacíos, DNI, teléfono, tipo y nombre inválidos	Aprobado
CP07–CP09	11:49 Errores y demora al asignar número	Primer código; siguiente a 001255 = 001256; 50 códigos sin repetir	Aprobado

CP10–CP12, CP22	11:40 Acumulación de solicitudes	Niveles Crítico, Alto, Medio y Bajo	Aprobado
CP13–CP15	11:54 Consulta de estado	Búsqueda existente, con minúsculas y no encontrada	Aprobado
CP16–CP17	Organización de registros	Orden por código y lista vacía	Aprobado
CP18–CP19	Pérdida de información	Archivo inexistente; guardar y recuperar	Aprobado
CP20–CP21	11:49 Duplicidad	Duplicado detectado; otro tipo no es duplicado	Aprobado

En la ejecución del menú, el registro asignó MOSA-2026-000001 con prioridad Crítico, rechazó un duplicado y datos inválidos, encontró el expediente por código y, al reiniciar el programa, recuperó los expedientes ordenados desde el CSV.

```
Solicitud registrada correctamente
Código: MOSA-2026-000001
Prioridad: Crítico (preliminar)

CÓDIGO            PRIORIDAD  ESTADO      SOLICITANTE
MOSA-2026-000001  Crítico    Registrado  Elber Nuñez Iguavel
MOSA-2026-000002  Medio      Registrado  Angel Rafael Caldas Ocmin
```

Errores detectados y corregidos

- **Duplicados:** el paso 2 exigía evitarlos pero solo había código único. Se agregó buscar_duplicado() (CP20 y CP21).
- **Palabra clave:** "niño" no detectaba "niña". Se usó la raíz "niñ" (CP22).

Limitaciones: "no hay heridos" se clasifica Crítico (se prefiere sobreclasificar); el DNI solo se valida por formato; el CSV no es multiusuario ni cifrado; el estado no cambia después de "Registrado".

Eficiencia: con 5 000 expedientes, ordenar tomó 0.64 s y buscar 0.29 ms; el prototipo responde de inmediato en este volumen.

Reflexión técnica y ética

- **Técnica:** dividir en funciones permitió probar cada regla por separado; contrastar el código con el paso 2 reveló la omisión de duplicados.
- **Datos personales:** la Ley N.º 29733 protege los datos personales. Las pruebas usaron datos ficticios y el CSV se excluyó del repositorio (.gitignore).
- **Priorización:** es preliminar, puede equivocarse y no reemplaza a los equipos de emergencia.
-

ANEXOS:



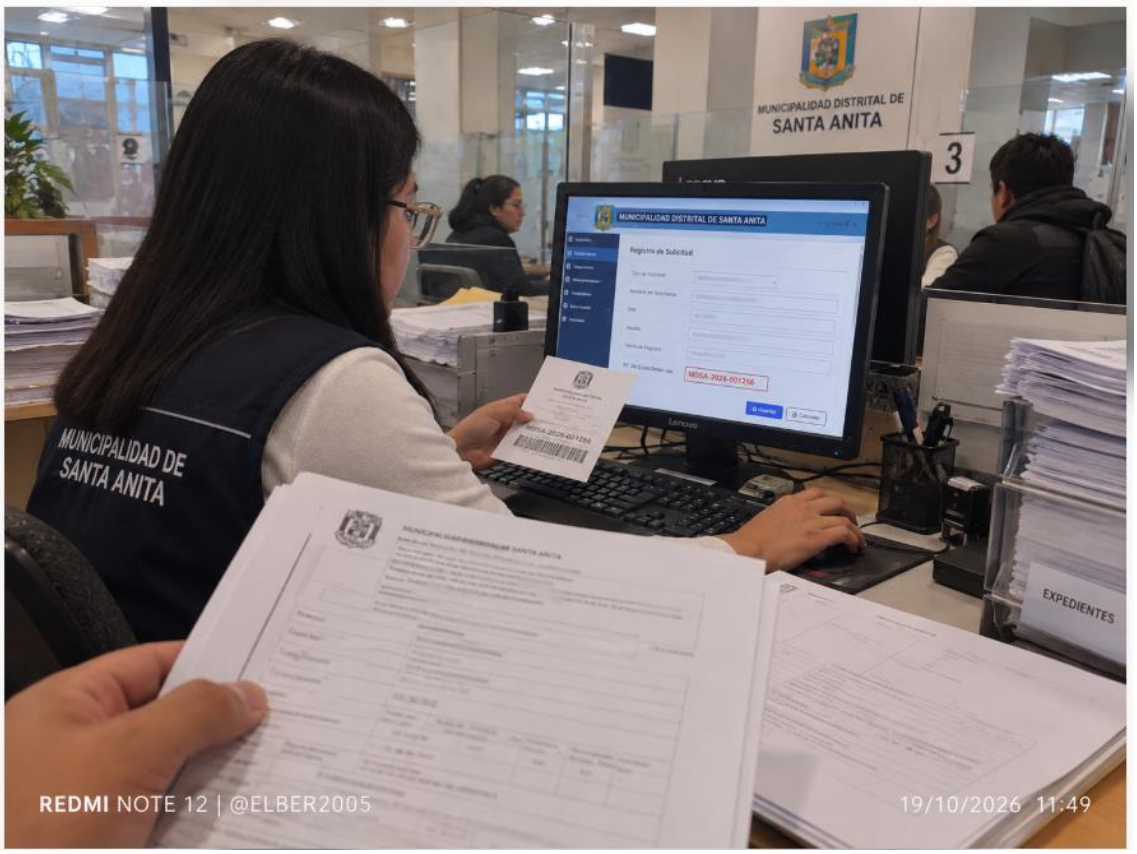
Anexo: 01



Anexo: 02



Anexo: 03



Anexo: 04



Anexo: 05

